# Android

3.

Bolla, Kálmán

# Constraint Layout

# Responsive UI

- To create a complex interface, we had to nest several layout (ViewGroup) elements
- The result is that the xml code describing the interface is complex, if we were to use a tree data structure to represent the relationship of the elements to each other, the depth of the tree gets deeper as the complexity increases
- The deeper the tree describing the interface, the longer it takes the system to load the interface
- Our goal is to create a surface of the same complexity while keeping the depth in check

# Responsive UI

- The previous problem (many parent-child relationships) was partially solved by using Relative Layout, but the real solution will be provided by one of Google's latest developments, Constraint Layout
- Another typical problem in development is that we need to support multiple resolutions, screen sizes
  - In the worst case we need more layouts, in the best case we just need more resources
- Constraint Layout will help us in this case too

# Constraint Layout

- Latest interface descriptor that allows the creation of complex interfaces using visual tools
- Android Studio's Layout Editor gets a bigger role, Constraint Layout is much easier to handle in design mode than writing XML by hand
  - For previous layout elements we preferred to work in XML view and check it later on the device or in design view
  - With this new layout element, the direction has been reversed, we implement the interface in design view, we prefer to use XML for post-processing

# Constraint Layout

- Supported from Android 2.3 (API level 9) → can be used on almost all existing Android devices
- When adding a new layout, Contraint Layout will be the parent element
- You can check in build.gradle file that it is available in your project:

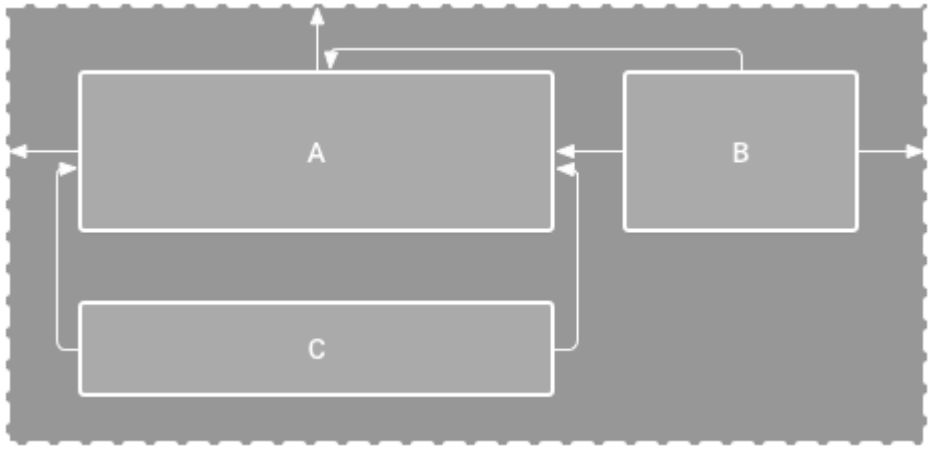
```
implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
```

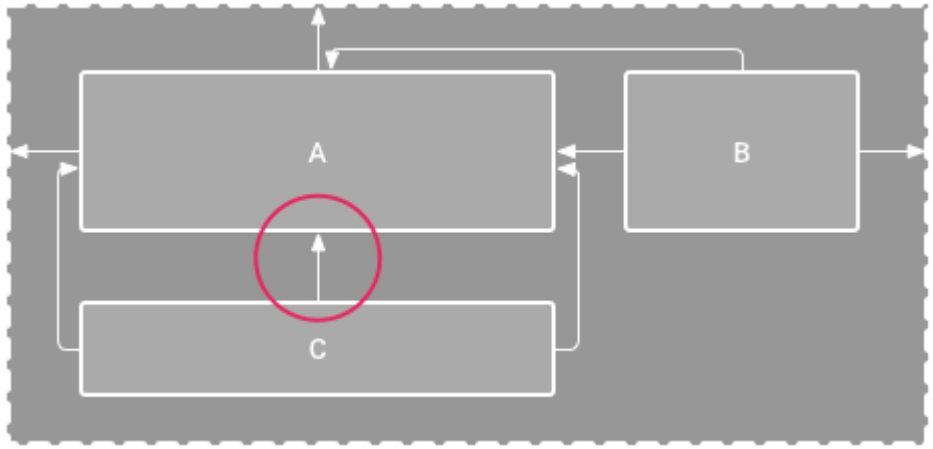
# Constraint Layout

- Constraint Layout now has the properties of Relative Layout, you can place the controls relative to each other on the interface
- A control (View) must have at least one horizontal and one vertical rule (constraint) defined
- The rules are a relationship or placement to another control, the parent layout or a guideline (a vertical or horizontal line that does not appear on the surface and to which we can align)

# Constraint Layout

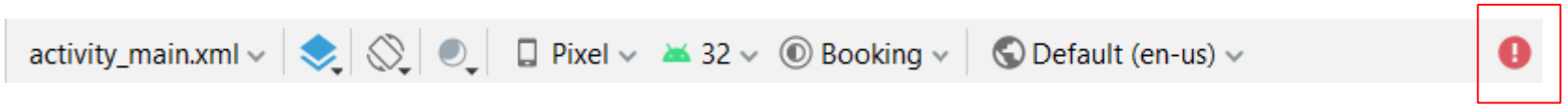
- Items dragged from the toolbar to the interface do not have constraints
  - If we run our application on a device in this state, the control will be moved to position 0,0 (top left corner)
- Example: horizontal rule for element C is given but due to the lack of vertical rule it appears on the full screen



# Constraint Layout

- If a rule is missing, you won't get a translation error, but an icon will appear on the toolbar to warn you that you forgot something




**Missing Constraints in ConstraintLayout**

This view is not constrained vertically: at runtime it will jump to the top unless you add a vertical constraint

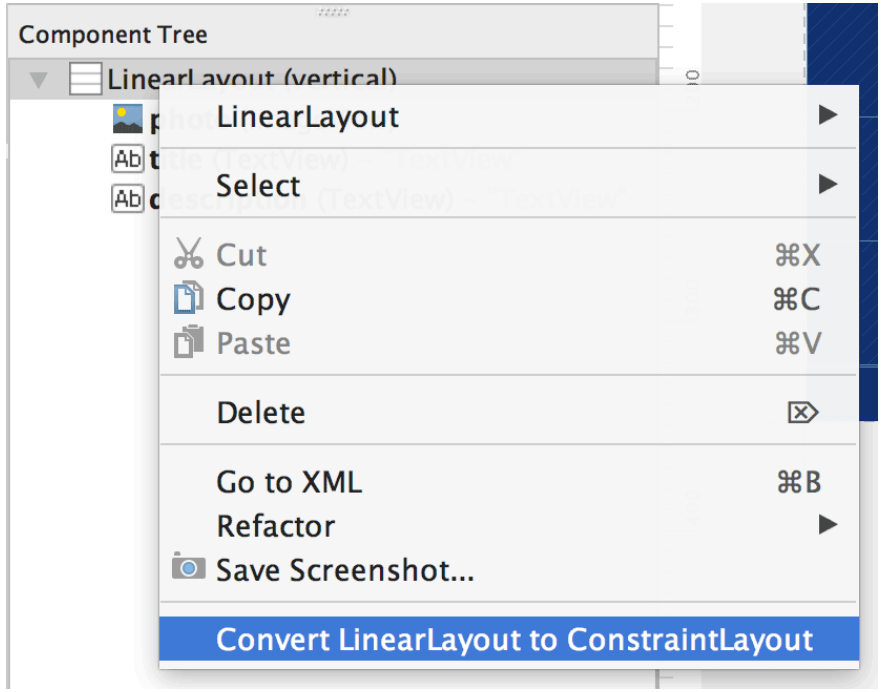
The layout editor allows you to place widgets anywhere on the canvas, and it records the current position with designtime attributes (such as `layout_editor_absoluteX`). These attributes are **not** applied at runtime, so if you push your layout on a device, the widgets may appear in a different location than shown in the editor. To fix this, make sure a widget has both horizontal and vertical constraints by dragging from the edge connections.

*Issue id: MissingConstraints*

Vendor: Android Open Source Project
Contact: https://groups.google.com/g/lint-dev
Feedback: https://issuetracker.google.com/issues/new?component=192708

# Convert an existing user interface

- For an existing project where an older type of layout element (e.g. Relative Layout) was used, Android Studio provides a conversion function
- Right-click on an interface to select this function



# Creating new layout

- With the new interface, we now get Constraint Layout as the root element by default

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

</androidx.constraintlayout.widget.ConstraintLayout>
```

# Add or remove constraint

- Designer interface showing the type of controller and its frame
- A circle on each page that allows you to create or cancel the rule
- You can drag the rule circle to another control or parent element to create a horizontal or vertical constraint
- In the case of an existing one, an **x** appears in a circle to delete the link

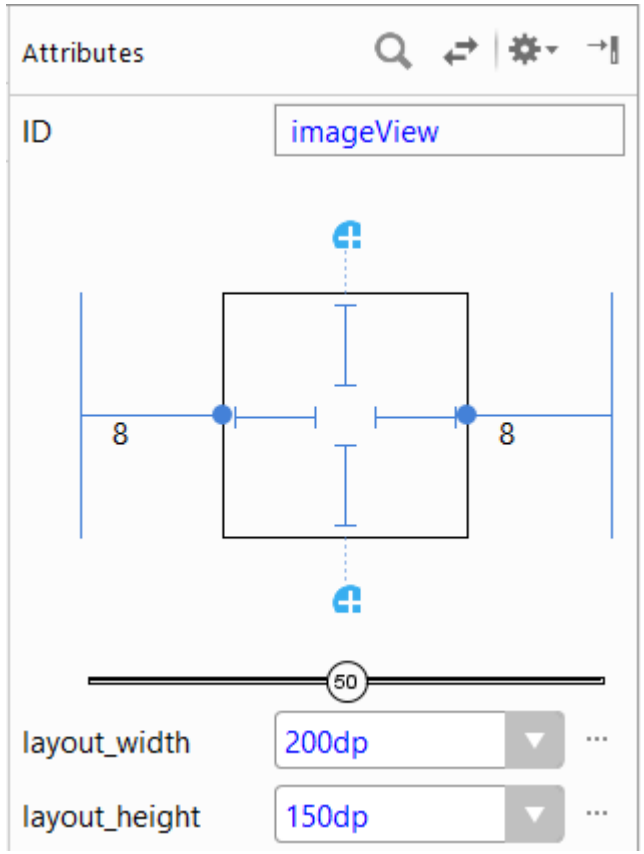
https://storage.googleapis.com/androiddevelopers/videos/studio/studio-constraint-first.mp4

# Rules

- When establishing connections between controllers, the following rules should be taken into account:
  - Each control (View) must have at least two constraints set (one horizontal and one vertical)
  - Horizontal can only be connected to horizontal, vertical to vertical (Baseline can only be connected to one baseline)
  - Each constraint can only handle one constraint

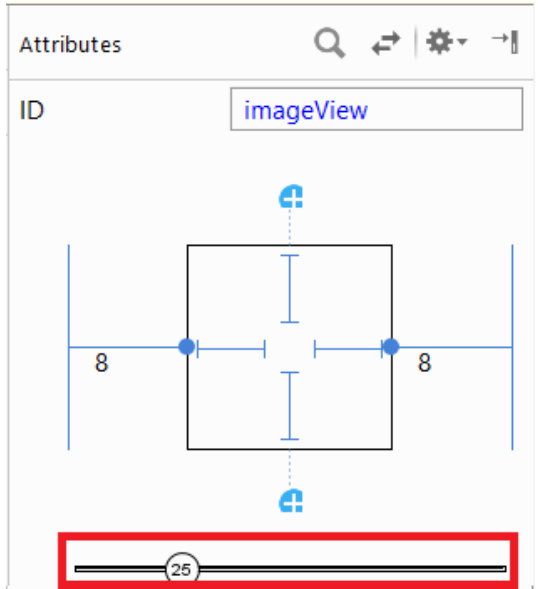
# Opposing constraints

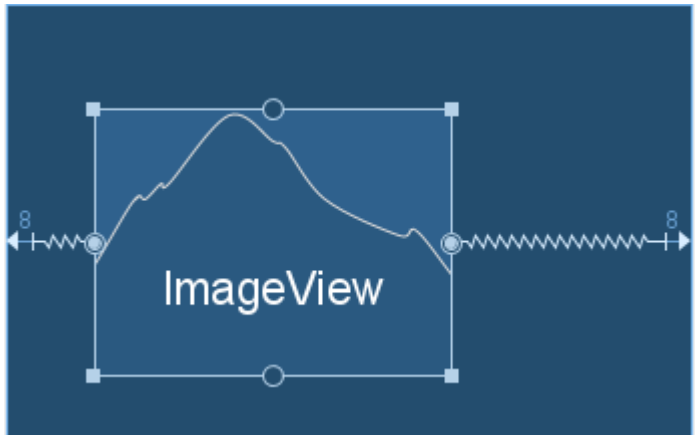
- If you set two opposite constraints the View will be centered (if the View size is fixed or set to wrap_content)




# Bias

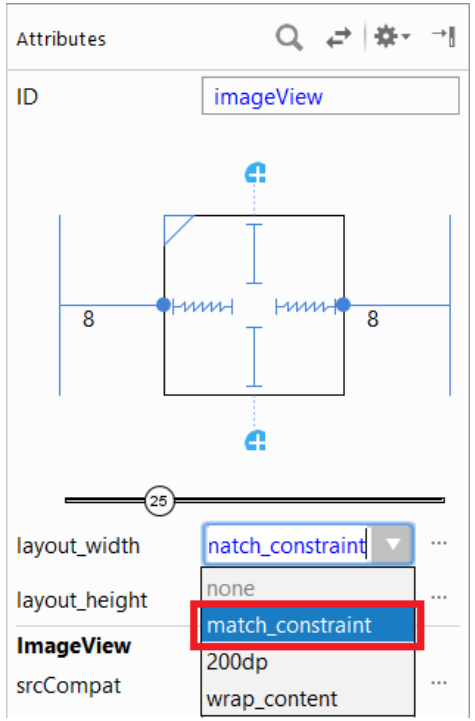
- Using Bias not only to center the View
- Xml:
  - *layout_constraintHorizontal_bias* and *layout_constraintVertical_bias* can take values between 0 and 1 (1 → 100%, 0.5 → 50%)
  - In the example, we set its value to 25 (0.25 in xml)



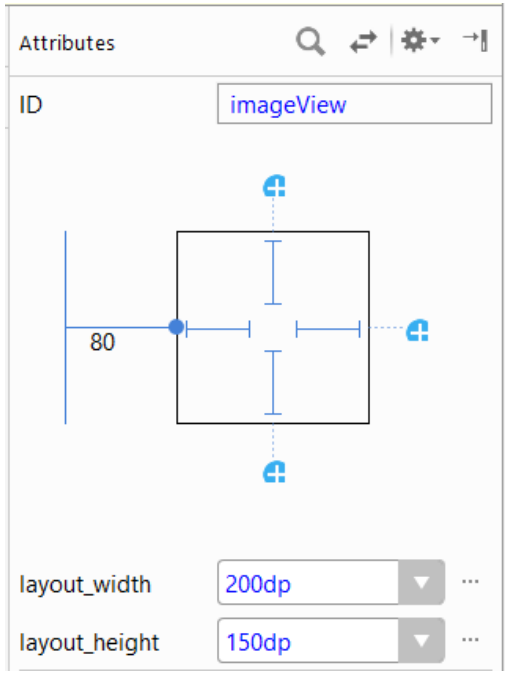
# Match constraints

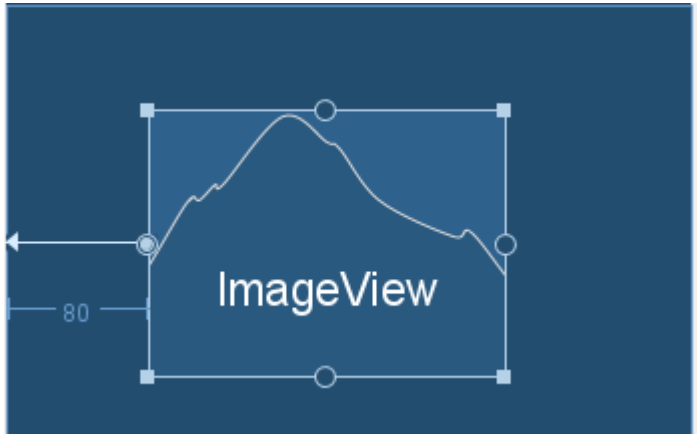
- You can also fill the available space by setting the View size to *match_constraints*




# Positioning to the parent

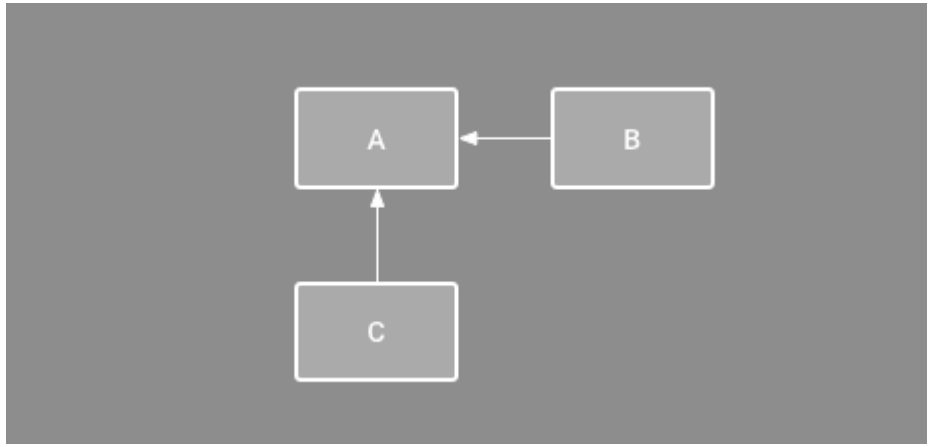
- You can align the control to one of the pages of the layout containing it
- Distance in dp units can be specified using margin
- In the example, the left side of the View is aligned to the left edge of the parent layout



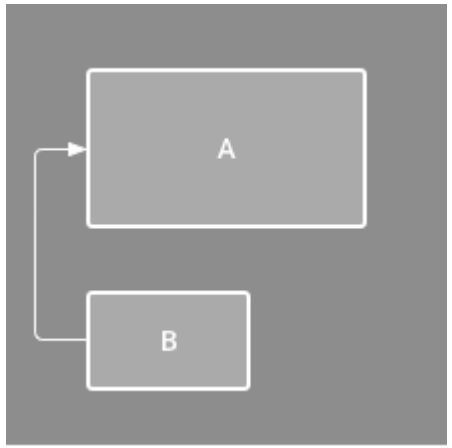
# Relative positioning

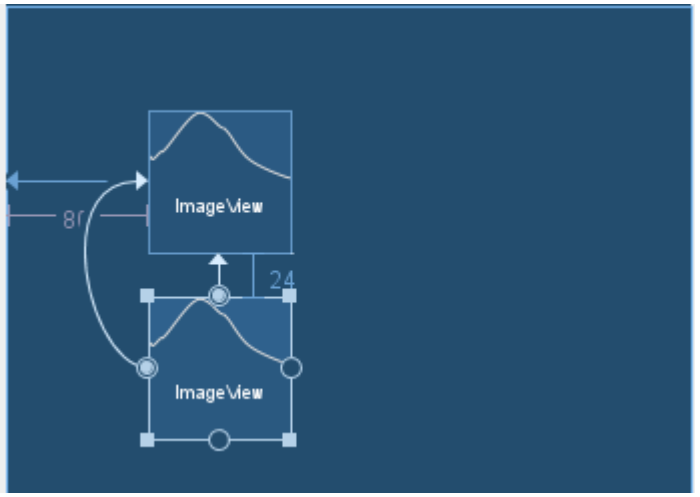
- Example: B is constrained to always be to the right of A, and C is constrained below A. However, these constraints don't imply alignment, so B can still move up and down

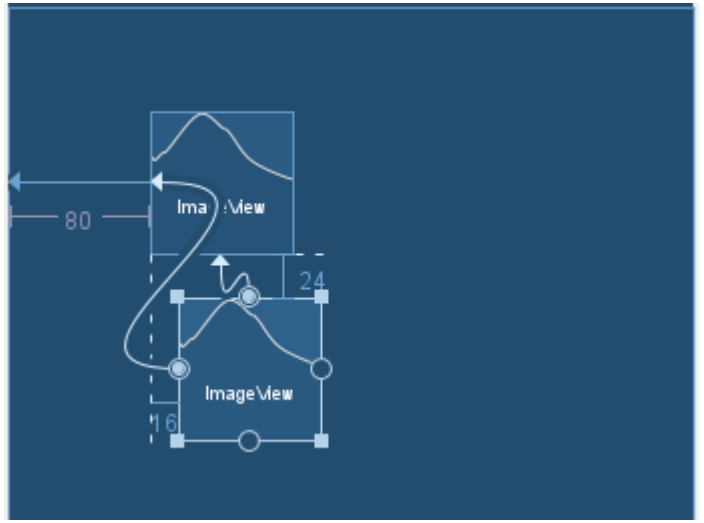


# Alignment

- Align the edge of a view to the same edge of another view
- The left side of B is aligned to the left side of A. If you want to align the view centers, create a constraint on both sides
- You can offset the alignment by dragging the view inward from the constraint. The offset is defined by the constrained view's margin

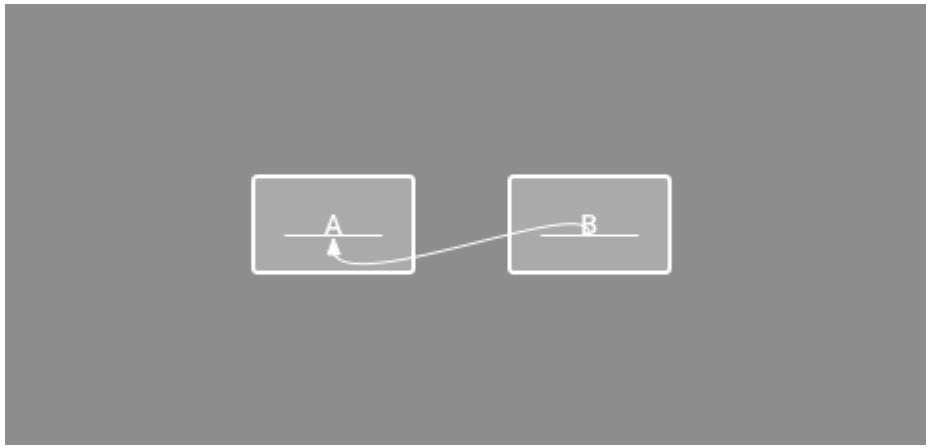


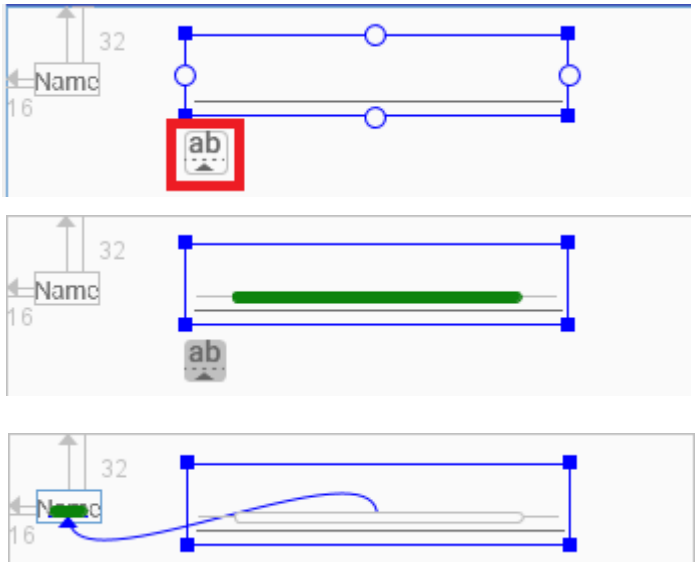



# Baseline alignment

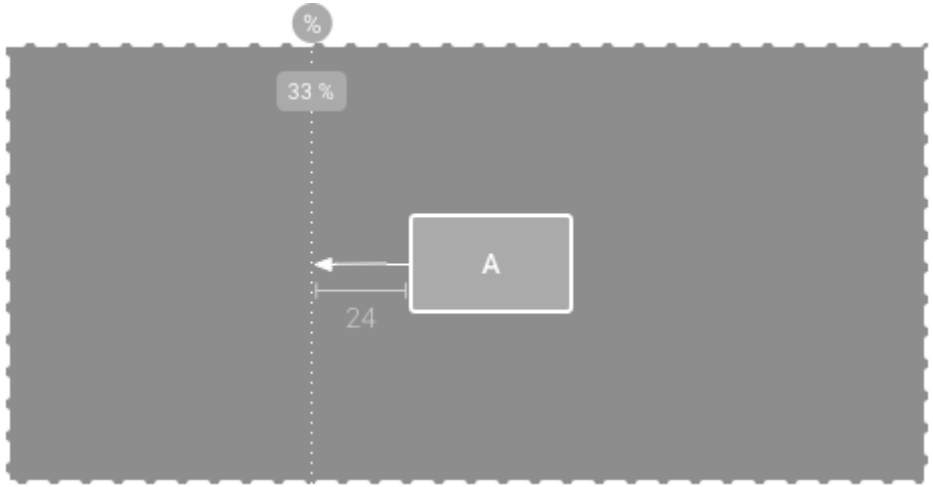
- Align the text baseline of a view to the text baseline of another view.
  - Example: the first line of B is aligned with the text in A
- To create a baseline constraint, right-click the text view you want to constrain and then click Show Baseline. Then click on the text baseline and drag the line to another baseline

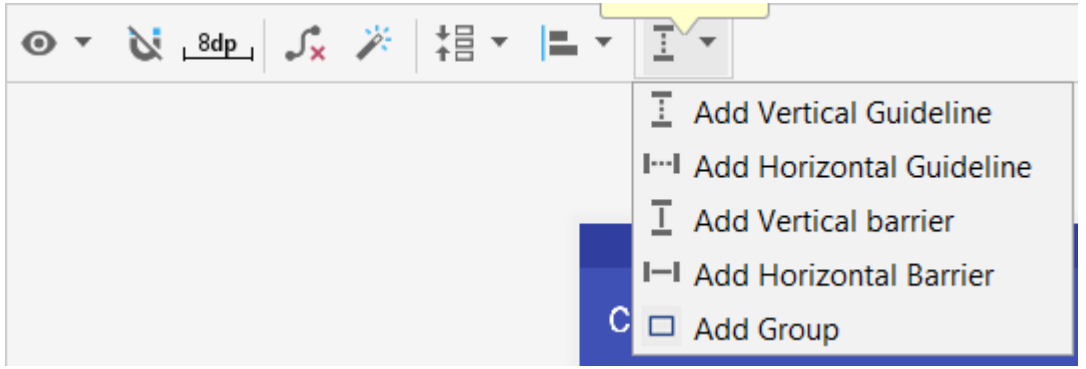


# Guideline

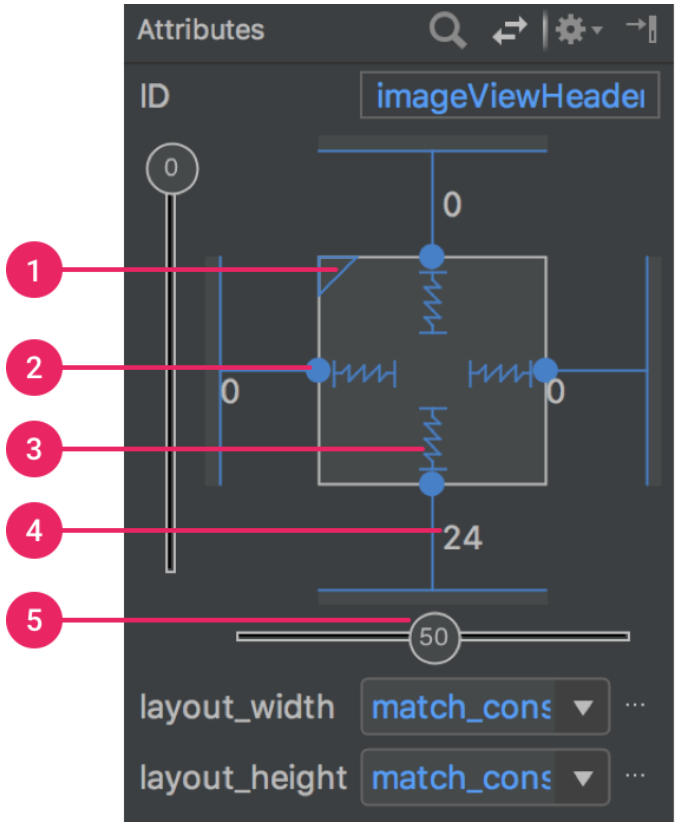
- You can add a vertical or horizontal guideline that lets you constrain your views and is invisible to your app's users. You can position the guideline within the layout based on either dp units or a percentage relative to the layout's edge
- Its value can be given in dp or % format



# Adjust the view size

- Attributes window:
- 1: size ratio
- 2: deleteing constraints
- 3: height or width mode
- 4: margin
- 5: bias

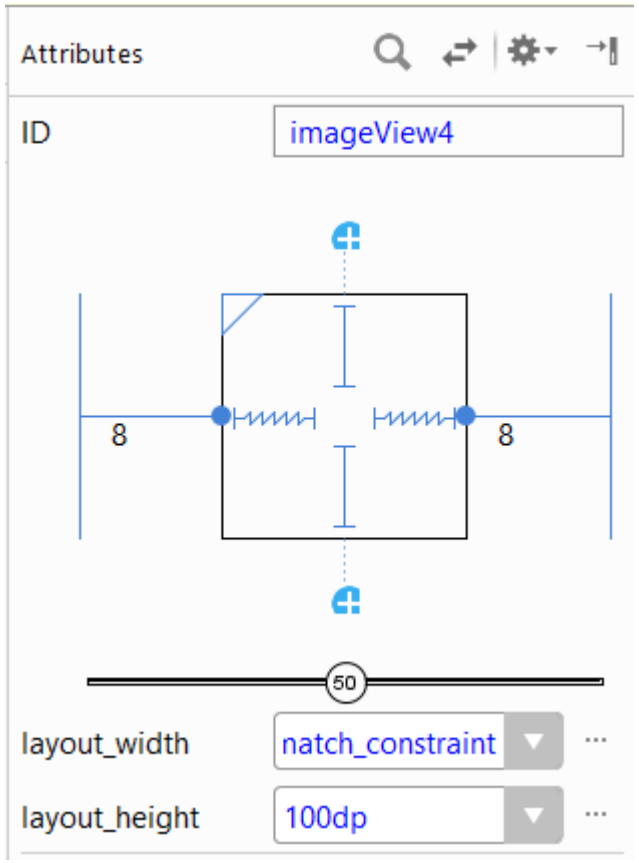


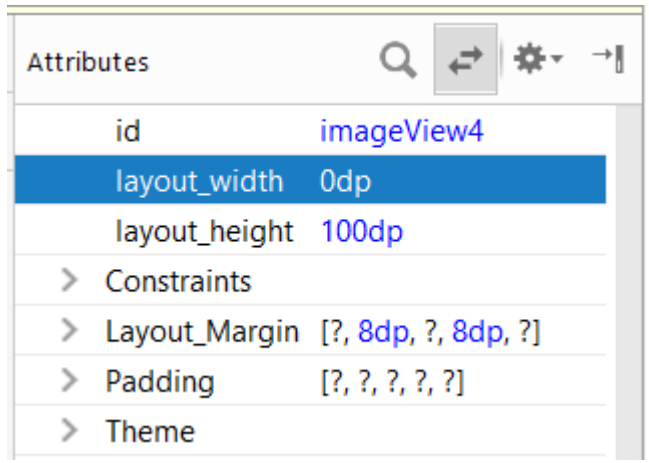
# View size

- **Fixed**: specify a specific dimension in the following text box or by resizing the view in the editor
- **Wrap Content**: the view expands only as much as needed to fit its contents.
- **Match Constraints**: the view expands as much as possible to meet the constraints on each side, after accounting for the view's margins. However, you can modify that behavior with the following attributes and values

# View size

- For match constraints, notice that the View size (layout_width or layout_height) defaults to 0dp

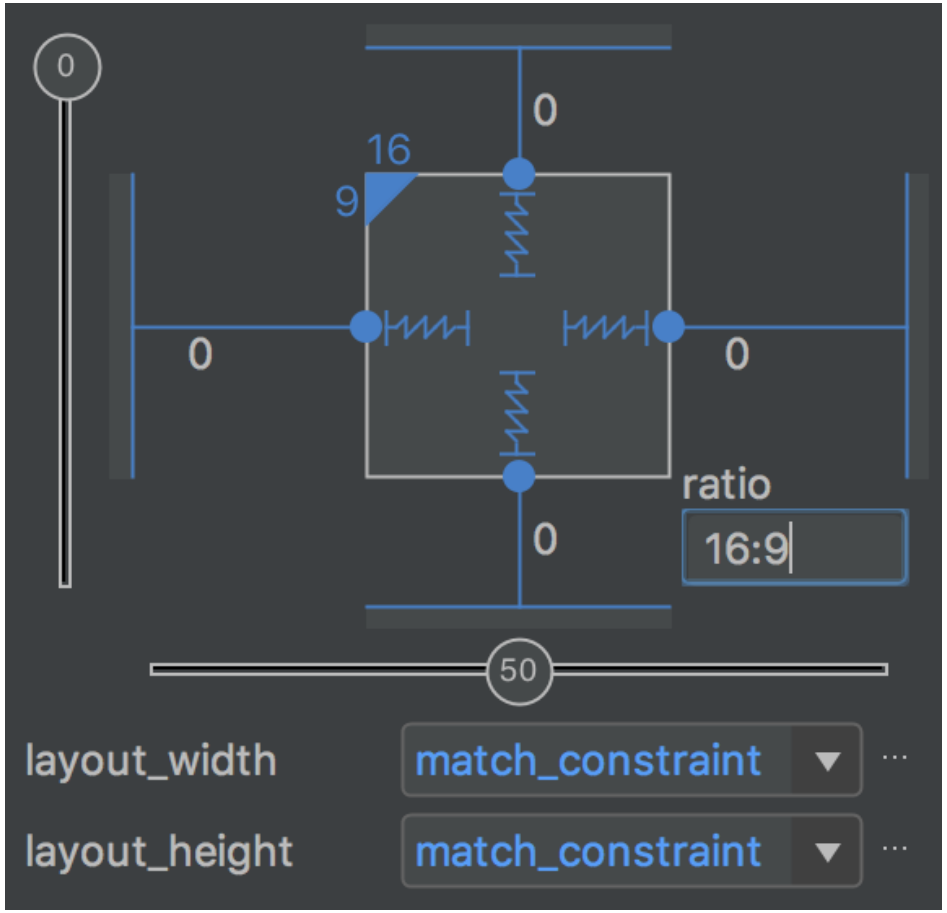



# View size

- For match constraints, we have additional options:
  - **layout_constraintWidth_default**
    - **spread**: as far as possible pulled to two sides (default behaviour)
    - **wrap**: is similar to the wrap_content setting, because it takes up as much space as the content requires, but may be smaller than the size of the control!
  - **layout_constraintWidth_min**: This takes a dp dimension for the view's minimum width
  - **layout_constraintWidth_max**: This takes a dp dimension for the view's maximum width

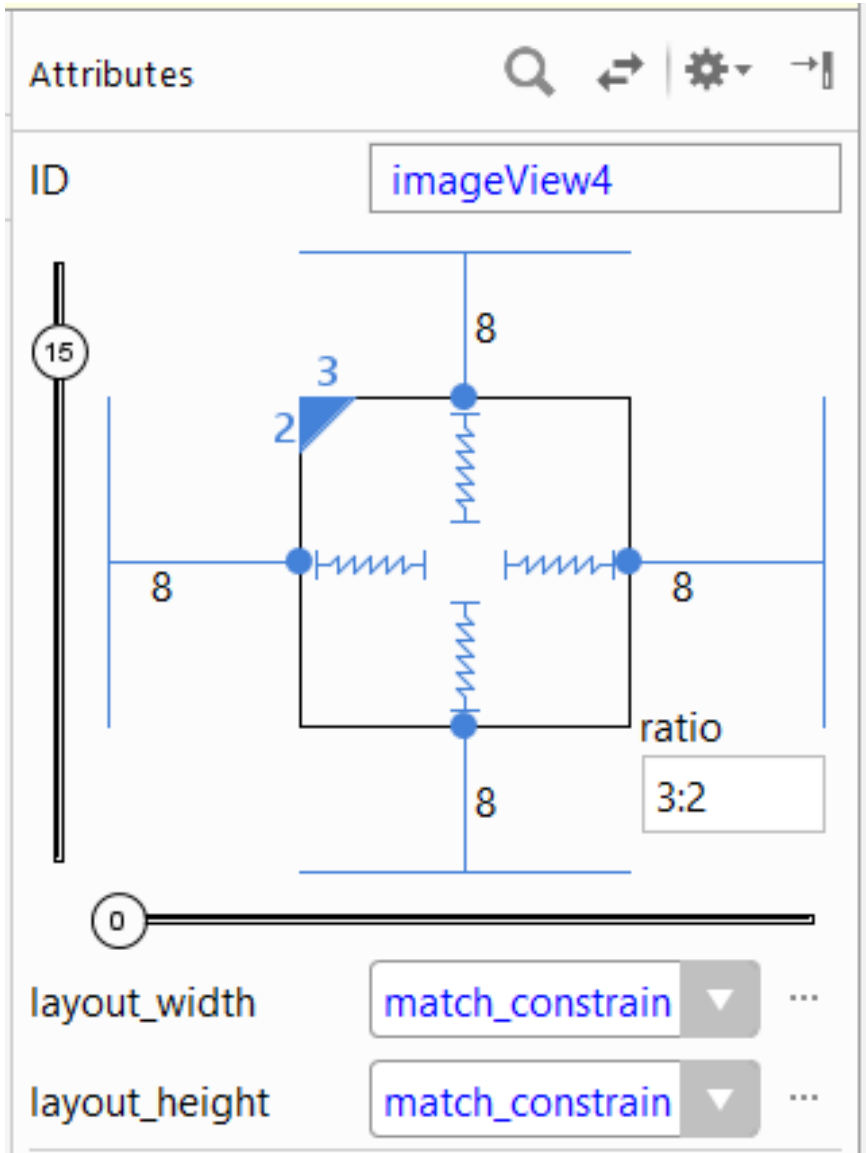
# Ratio

- If at least one of the View dimensions is set to match_constraints the size ratio
- E.g.: can be set to 16:9



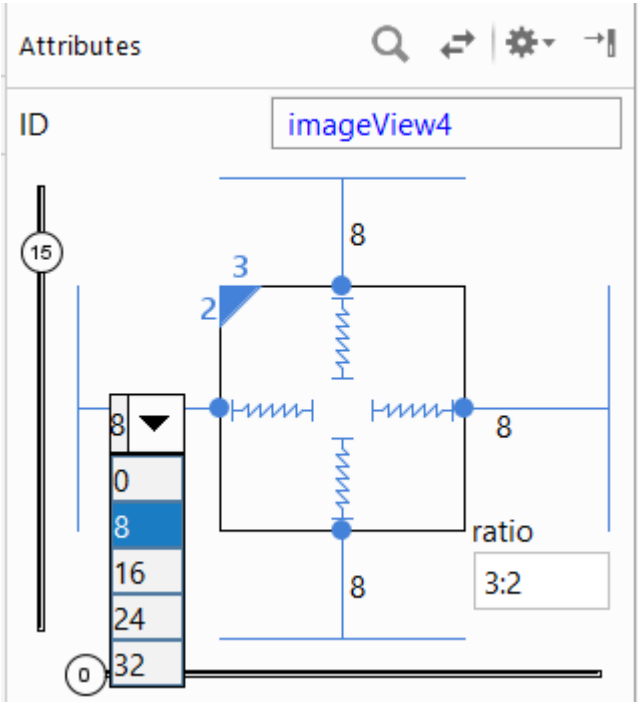
# Ratio

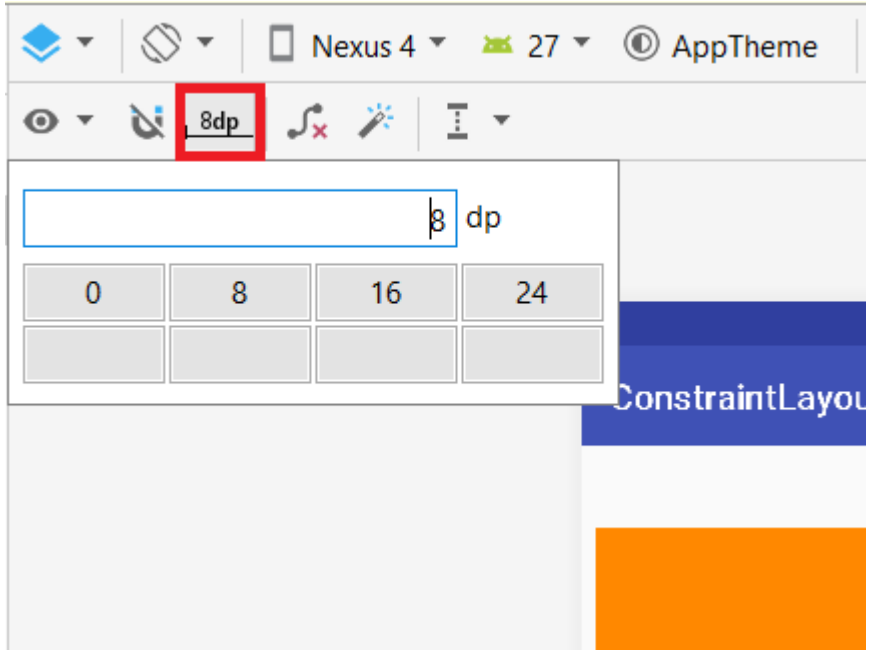

8
ImageView
8


Attributes
ID
imageView4
15
8
3
2
8
8
ratio
3:2
8
0
layout_width
match_constrain
layout_height
match_constrain

# Margin

- Margin can be set from the Attributes window
- The default margins for all controls can be found in the top row of the designer



# Automatic positioning

- Android Studio's designer also gives you the option to set the position of the Views one by one instead of manually
- Place the elements on the interface (where you want them to appear later) and select Infer Constraints
  - The development environment will attempt to automatically calculate and set the layout of the views placed on the interface within the Constraint Layout
  - Depending on screen size and resolution, some post-processing may be required

# Additional features

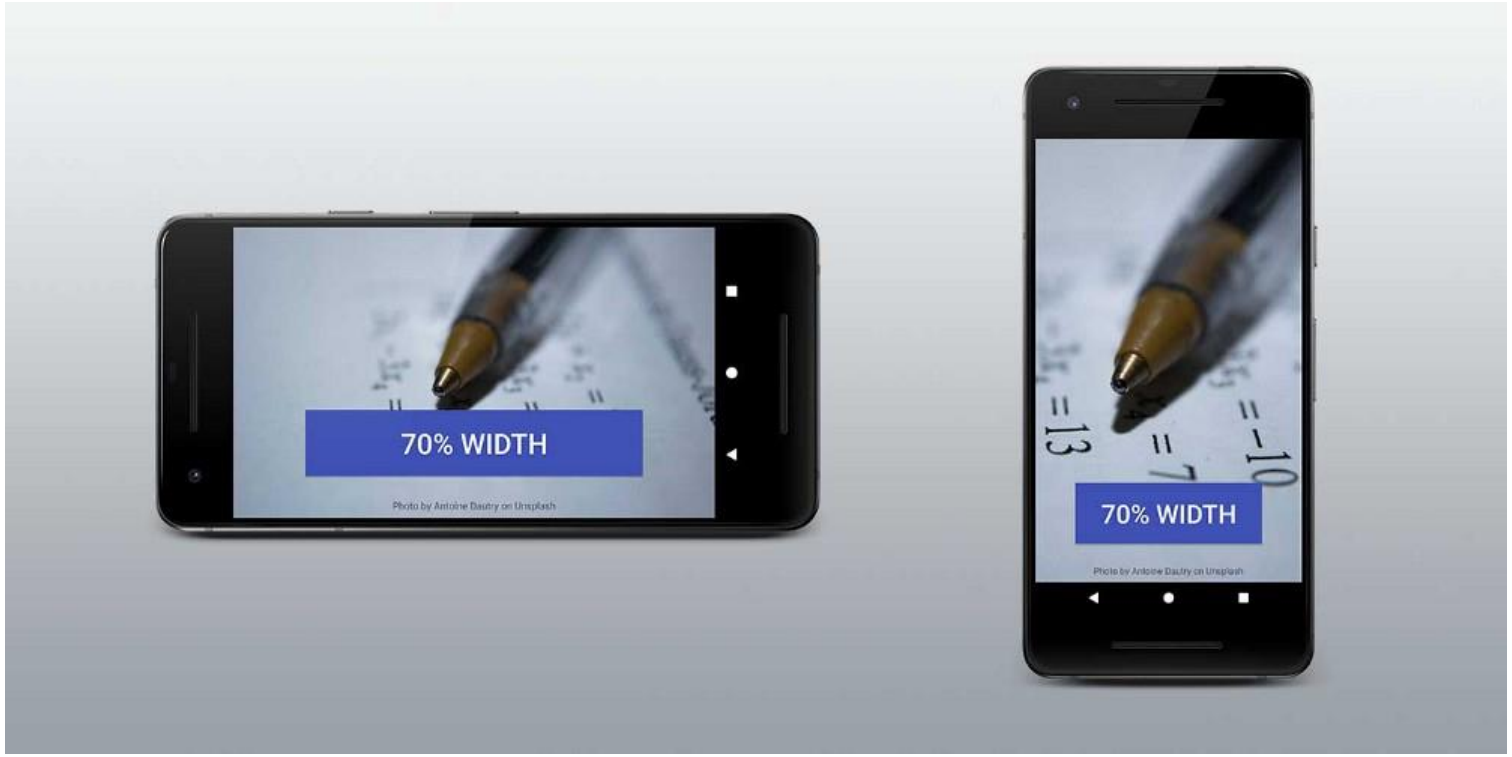
# New features

- Constraint Layout is constantly being improved, giving us more options for the placement of controls
- In Android Gradle (build.gradle) you can check which version you are using

```
implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
```

# Percents

- If we wanted to specify the size of a View in %, we had to bind 1-1 guideline (also in %) to the two opposite sides



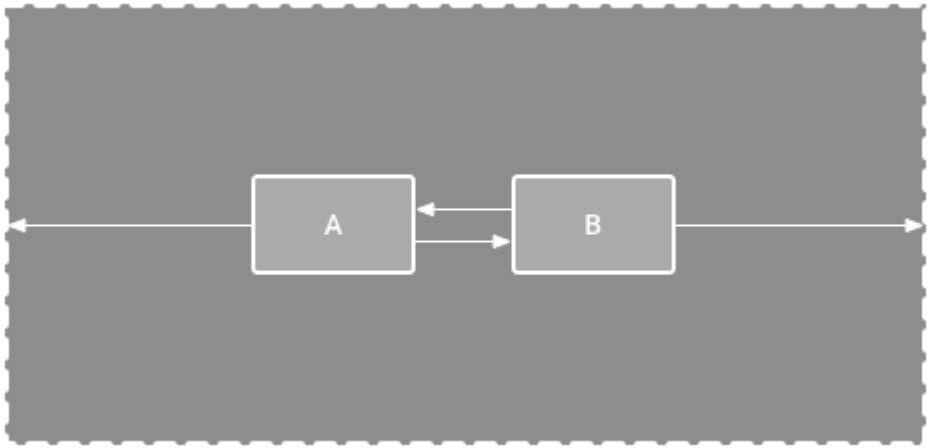
# Percents

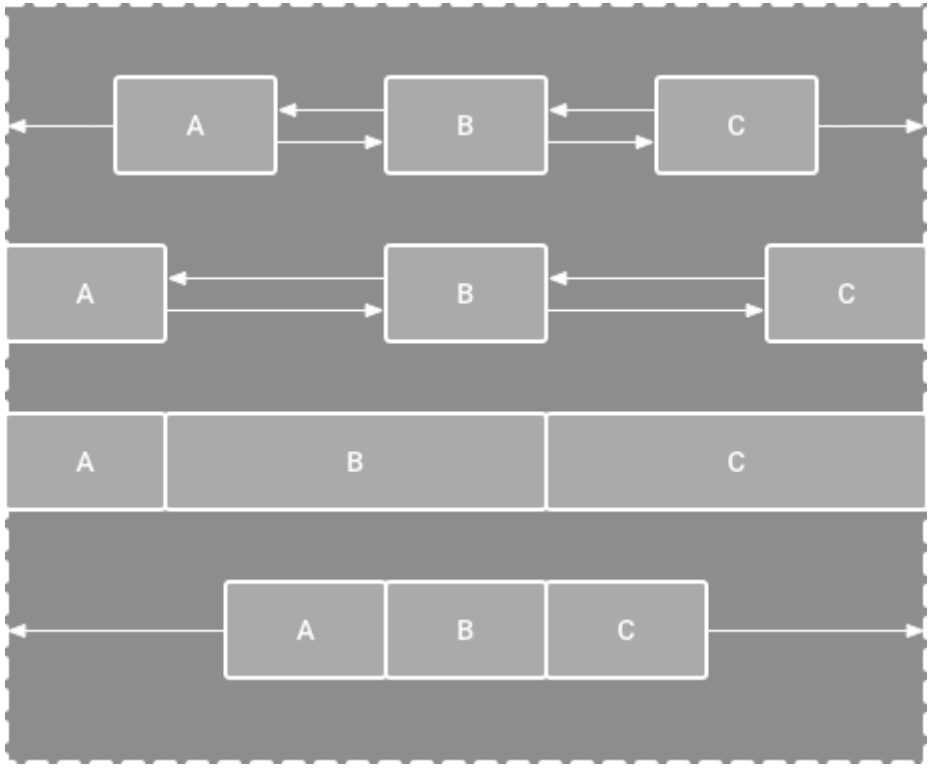
- Each View has the possibility to specify its width, height in %
  - Fills the available space in the given proportion
- XML attributes:
  - *layout_constraintWidth_percent*
  - *layout_constraintHeight_percent*

```
<Button
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    app:layout_constraintWidth_percent="0.7" />
```

# Chain

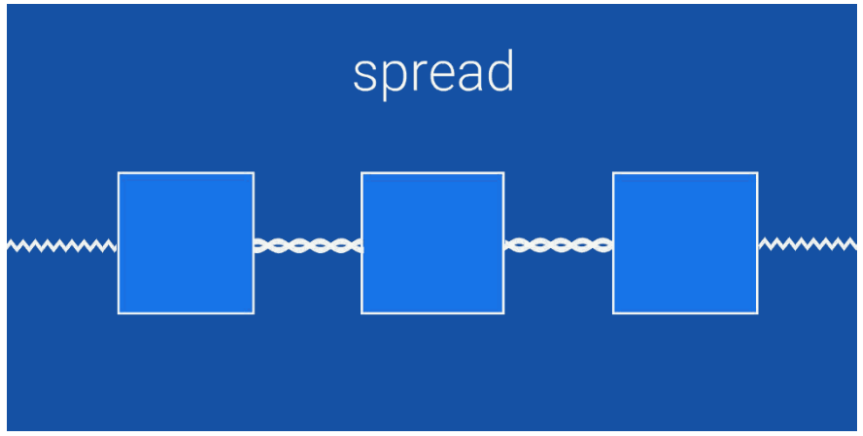
- A chain is a group of views that are linked to each other with bi-directional position constraints. The views within a chain can be distributed either vertically or horizontally

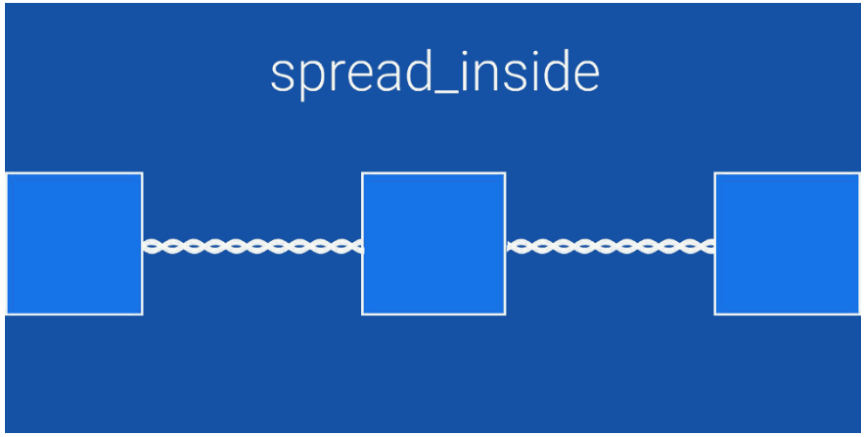


# Chain

- Chains can be styled in one of the following ways:
  - **Spread**: the views are evenly distributed after margins are accounted for. This is the default.
  - **Spread inside**: the first and last views are affixed to the constraints on each end of the chain, and the rest are evenly distributed



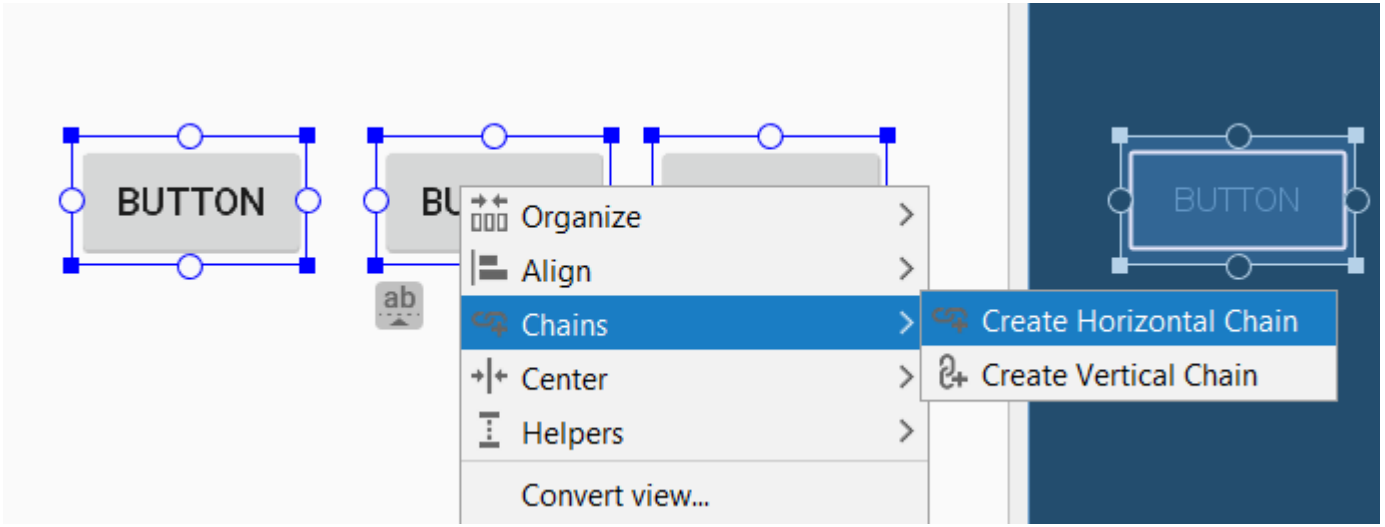



# Chain

- **Weighted**: when the chain is set to spread or spread inside, you can fill the remaining space by setting one or more views to match constraints (0dp)
  - You can assign a weight of importance to each view using the layout_constraintHorizontal_weight and layout_constraintVertical_weight attributes
  - This works the same way as layout_weight in a LinearLayout
- **Packed**: the views are packed together after margins are accounted for

# Chain

- Create chain constraints:
  - https://storage.googleapis.com/androiddevelopers/videos/studio/constraint-chains.mp4

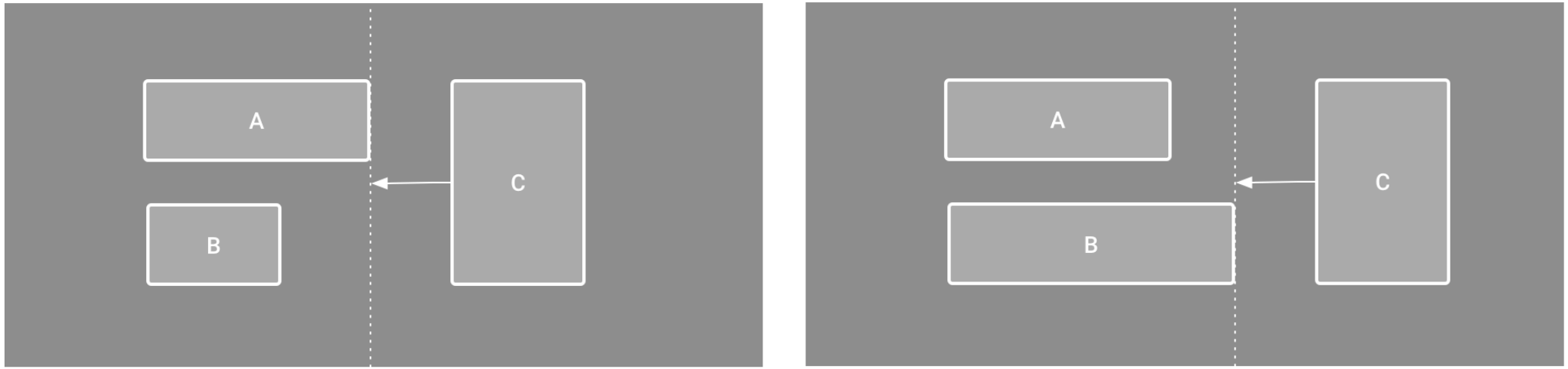


# Chain

- After selecting controls, right-click to access the shortcut menu where you can set the chain:
  - Chains:
    - Create Horizontal Chain
    - Create Vertical Chain
- Different display modes can be accessed by repeatedly pressing the chain button (  )

# Barrier

- Similar to a guideline, a barrier is an invisible line that you can constrain views to, except **a barrier doesn't define its own position**
- Useful when you want to constrain a view to a set of views rather than to one specific view
- Example: the barrier position is depending on view A and view size



# Groups

- Sometimes we need to hide or display several items at once in the user interface
- Controls can now be grouped together, which solves the previous problem
- Group does not affect the View hierarchy, you can define one or more members in an attribute

# Groups

- In the example, we create a group with profile ID and assign *profile_name* and *profile_image* Views to the group

```
<androidx.constraintlayout.widget.Group
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:id="@+id/profile"
   app:constraint_referenced_ids="profile_name,profile_image" />
```

- You can set the group to appear or disappear by code:

```
Group profile = findViewById(R.id.profile);
// hide profile_name, profile_image views
profile.setVisibility(View.GONE);
// show profile_name, profile_image views
profile.setVisibility(View.VISIBLE);
```

# Circular Constraints

- Constraint Layout korábbi verziójában csak vízszintes és függőleges irányban pozicionálhattunk, az új verzióban tetszőleges szögben, egy körvonalon helyezkedhet el egy vezérlő a másikhoz képest
- Vízszintes és függőleges margin helyett szöget és a kör sugarát paraméterezzük
  - A szög a vezérlő felett, óramutató járásával megegyező irányban állítható
- Látványos felület létrehozására használhatjuk, pl. saját menürendszer

# Circular Constraints

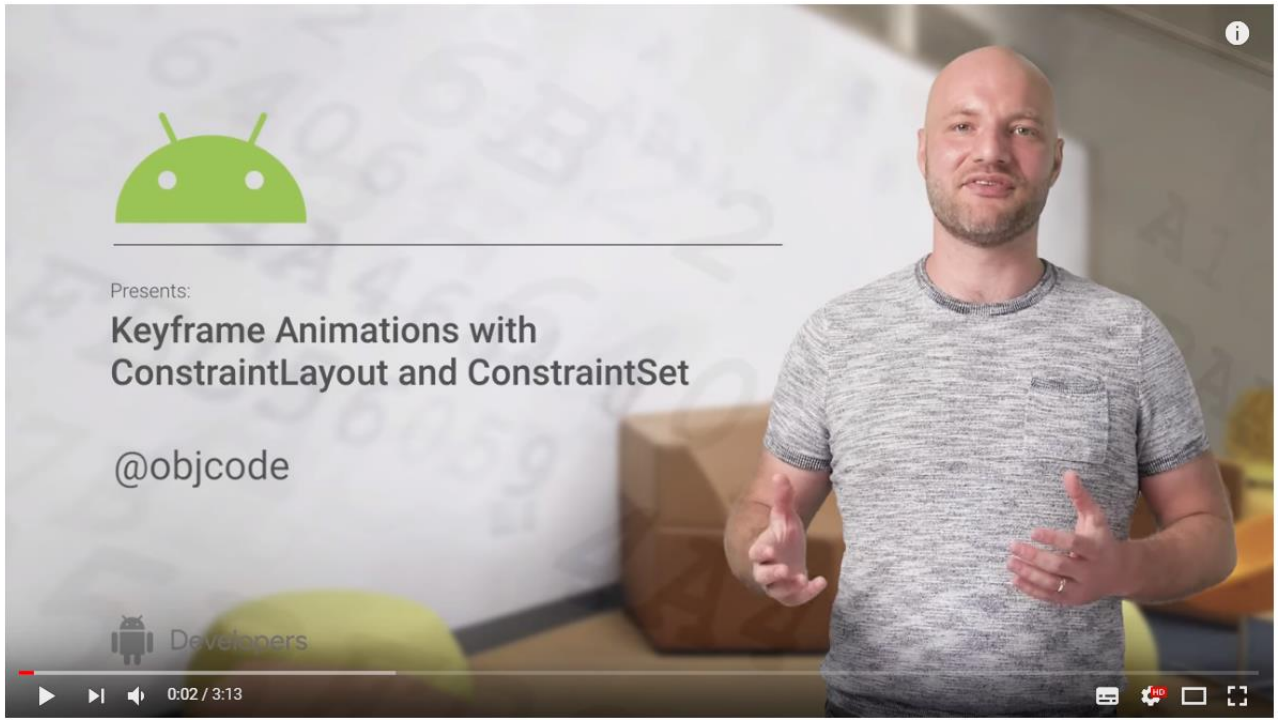
```
<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/middle_expanded_fab"
    app:layout_constraintCircle="@id/fab"
    app:layout_constraintCircleRadius="50dp"
    app:layout_constraintCircleAngle="315"
    tools:ignore="MissingConstraints" />
```

# Animations

- Constraint Layout also helps with animations, we can animate multiple views at once with minimal java/Kotlin code and xml writing
- Solution is, which contains the settings for a Constraint Layout
  - Can be created from code and layout file
- The animation can be played with the TransitionManager.beginDelayedTransition() function

# Animations

- Keyframe Animations with ConstraintLayout and ConstraintSet
  - https://www.youtube.com/watch?v=OHcfs6rStRo



# UI optimization

- Using too many constraints can also lead to slower surface loading, of course
- Optimization has been added to the new version to reduce them
- The layout_optimization Level attribute defines several optimization levels:
  - barriers, direct, standard, dimensions, chains

```
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    app:layout_optimizationLevel="standard|dimensions|chains"
```

# Resources

- Build a Responsive UI with ConstraintLayout
  - https://developer.android.com/training/constraint-layout/
- Introducing Constraint Layout 1.1
  - https://medium.com/androiddevelopers/introducing-constraint-layout-1-1-d07fc02406bc